AUDITORIA DE SEGURANÇA

Relatório de Auditoria de Segurança

Gabaritei Questões

Revisão de isolamento de dados, autorização, IDOR, segredos e XSS em uma aplicação Flask + Supabase + Electron.

1 ACHADO CRÍTICO

DATA **02/11/2026**	ESCOPO 11 blueprints, 80+ handlers, templates/JS, docs, .env, git e bundle Electron
STACK Python/Flask, Supabase/PostgREST, Jinja2, Tailwind/Bootstrap, Electron	RESULTADO 15 achados • 9 issues para o GitHub

Nota metodológica: cada categoria foi mapeada para o equivalente da stack detectada. A seção 3 detalha o escopo, as limitações e a cobertura rota a rota.

1. Resumo executivo

A auditoria estática identificou **15 achados: 1 crítica, 5 altas, 3 médias, 4 baixas e 2 informativas.** O risco central não é um endpoint isolado: é a combinação de uma policy de SELECT documentada como livre com uma anon key pública, que transforma a leitura de dados de aluno em um dump cross-tenant sem autenticação. Sobre essa base, foram confirmados IDORs em cadernos e simulados privados, um XSS armazenado em comentários e uma rota legada que confia no placar enviado pelo cliente.



Leitura rápida

• **O SELECT é a porteira aberta.** A documentação afirma que 13 tabelas de dados do aluno têm SELECT livre para anon; a anon key está no cliente. Sem login, é possível ler e-mails, histórico, comentários, cadernos, flashcards e simulados de todos.

• **Os IDORs sobrevivem ao RLS.** Mesmo que o SELECT seja fechado, cadernos e simulados privados continuam acessíveis por ID porque as rotas não comparam o dono.

• **O XSS é armazenado e cross-user.** Texto e nome de usuário entram crus no innerHTML; o CSP com unsafe-inline permite o handler. O payload atinge qualquer leitor da questão.

• **O restante do backend tem boa disciplina.** CSRF global, aluno_required no servidor, filtros de dono na maior parte do CRUD e correção server-side na rota principal de resposta.

Condições de explorabilidade
A falha crítica é documental: as migrations de RLS não estão no repositório. Não foi executada consulta contra a produção; a exploração depende de a configuração descrita em docs/banco_dados.md estar vigente. Os IDORs exigem um aluno autenticado e IDs numéricos. O XSS exige que a vítima abra/expanda os comentários da questão. A rota /api/responder não é chamada pelo frontend atual, mas permanece acessível.

2. Visão quantitativa

Os gráficos abaixo usam a paleta definida para o relatório. A distribuição por severidade mostra o peso da falha crítica de RLS; a distribuição por categoria mostra que o IDOR é o grupo mais numeroso, seguido por isolamento e chaves.

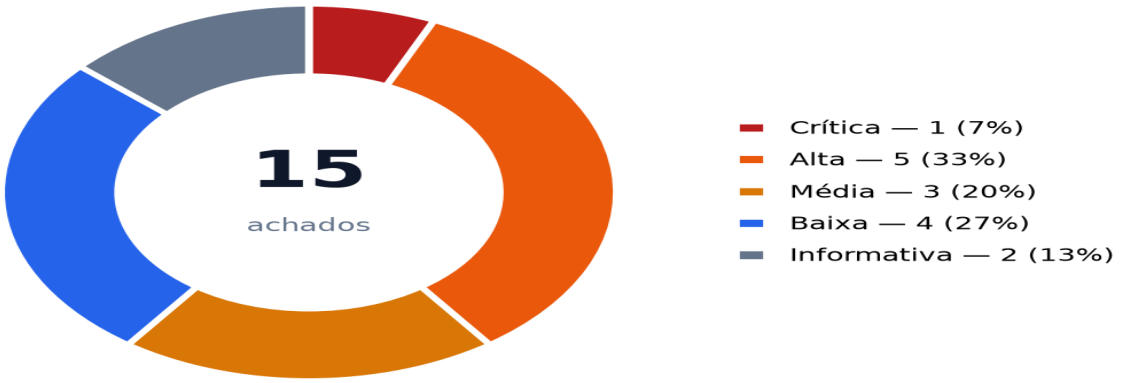


Figura 1 — Achados por severidade (total e percentual).

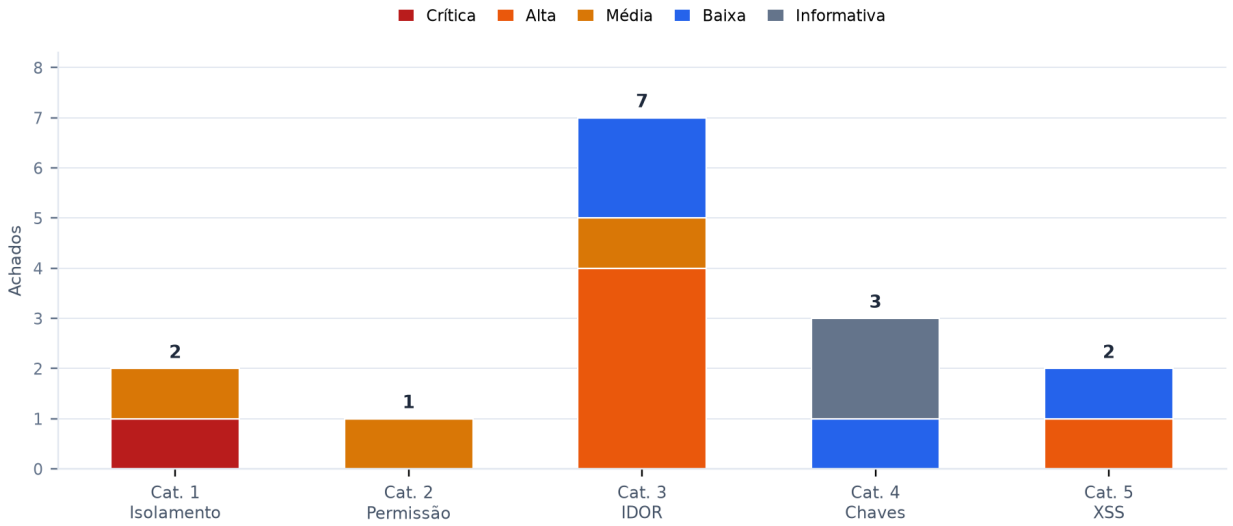


Figura 2 — Achados por categoria, empilhados por severidade.

Como ler: a categoria 3 (IDOR) concentra 7 dos 15 achados, mas a categoria 1 contém o único achado crítico e o de maior impacto sistêmico. A categoria 4 é majoritariamente informativa: a anon key é pública por design e o APP_TOKEN é extraível, mas nenhum dos dois, sozinho, permite escrever como outro aluno. A categoria 5 tem um XSS alto e um conjunto de sinks latentes.

3. Escopo e metodologia

A stack foi detectada antes da auditoria e cada categoria foi adaptada ao equivalente dela. Foram lidos todos os módulos de rota, os módulos de banco/utilitários, os templates e o JavaScript, a documentação de segurança e de rotas, o .env, o histórico git, o site público e o bundle Electron. Não foram executados exploits contra a produção nem consultas a dados reais de alunos.

Linguagem / framework	Python 3 + Flask; Flask-Login, Flask-WTF (CSRF), Flask-Limiter, Flask-Talisman.
Banco / query builder	Supabase PostgreSQL via PostgREST (SDK supabase/postgrest), ANON key + header x-app-token.
Frontend	Jinja2 + Tailwind/Bootstrap + JavaScript vanilla; marked.js, KaTeX, SweetAlert2.
Desktop	Electron (contextIsolation, sandbox, nodeIntegration false) + backend PyInstaller.
Deploy	Sem Docker/CI/Helm/Terraform no repositório; backend escuta 127.0.0.1:5000.

3.1 Mapeamento das cinco categorias

Categoria	Equivalente na stack	Como foi verificado
1. Isolamento de inquilino/dono	Supabase/PostgREST: policies RLS + filtros manuais por id_usuario nas rotas Flask.	Leitura das policies documentadas, de todas as queries e dos filtros por id_usuario; confronto com o modelo de dados.
2. Permissão definida no navegador	Flask-Login + aluno_required. Não existe UI administrativa neste repositório; o papel é fixado em 'aluno' no servidor.	Inventário de gates de papel no frontend e no backend; confirmação de que não há UI/admin neste repo; revisão de lógica de negócio confiada ao cliente.
3. IDOR	Rotas Flask que recebem id de objeto (path, query ou body) e deveriam validar o dono.	Leitura de todos os handlers das 11 blueprints; tabela de cobertura com cada rota que recebe ID ou muta dados.
4. Chaves expostas	.env, histórico git, bundle Electron/app.asar, site público e configuração de build.	Busca por chaves/tokens/senhas em código, configs, docs e bundle; git log/git grep; inspeção do app.asar; .env e .env.example.
5. Inputs sem tratamento (XSS)	Jinja2 + JavaScript vanilla: innerHTML, marked.parse, |safe, CSP e ausência de sanitizador.	Varredura de innerHTML, |safe, marked.parse, srcdoc e eval; verificação de sanitizador nas dependências; análise do CSP.

Categoria 2 — não se aplica à UI administrativa. Este repositório não contém tela, rota ou componente de administração; o papel é fixado em *aluno* no servidor (rotas/auth.py:36-41 e app.py:316-321). Por isso a categoria não rende achados de "permissão no navegador". O único achado associado é de lógica de negócio confiada ao cliente: **/api/responder** aceita *is_correct* do JSON (F-08).

3.2 Limitações e cobertura

• As migrations de RLS não estão neste repositório. A verificação do SELECT aberto é documental (docs/banco_dados.md:27,38) e não substitui uma consulta ao dashboard/SQL de produção.

• Não foi executada nenhuma consulta contra a produção nem usado dado real de aluno; a auditoria é estática e baseada no código/documentação do repositório.

• O painel administrativo vive em repositório privado separado (gabaritei-admin) e ficou fora do escopo. Este repo não contém rotas administrativas nem UI de admin.

• Não há Docker, CI, Helm ou Terraform no repositório; a análise de deploy se limita ao Electron, PyInstaller, .env e site público.

• A ausência de sanitizador foi confirmada em requirements.txt/package.json; a exploração de cada sink depende do grau de confiança do conteúdo que chega até ele.

4. O que foi verificado e está correto

A auditoria não se limita a defeitos. Os controles abaixo foram conferidos no código, na documentação ou no histórico do repositório e estão corretos no estado atual. Eles reduzem o impacto de parte dos achados e devem ser preservados nas correções.

#	Controle verificado	Evidência	Por que importa
1	**CSRF global em todas as rotas de escrita**	`extensoes.py:12`; `app.py:140`	CSRFProtect é inicializado globalmente e não há csrf.exempt no projeto. Formulários incluem csrf_token() e o AJAX envia X-CSRFToken (conta.html:307-385; questoes_common.js:27-163).
2	**Autorização de papel no servidor, não no navegador**	`utils.py:63-70`; `app.py:316-321`; `rotas/auth.py:36-41`	aluno_required valida is_authenticated e role == 'aluno' no backend; o papel é fixado em 'aluno' no load_user e no login. Não existe UI administrativa nem rota privilegiada neste repositório.
3	**Correção de respostas calculada no servidor**	`rotas/questao.py:33-47`; `rotas/edital.py:493-501`	A rota principal /aluno/responder e responder_edital buscam as alternativas e comparam a letra no servidor; o cliente não decide o acerto (a exceção é a rota legada /api/responder, achado F-08).
4	**Filtros de dono na maior parte do CRUD**	`caderno.py:16,80,138-141,168-174`; `flashcards.py:17-19,43,52-53,78,162-167`; `resumos.py:14,40,50,60`; `filtro.py:24,45`; `edital.py:291,384-393,485-490,603-610,717`	Listagens, exclusões e updates de cadernos, flashcards, resumos, filtros, editais e simulados aplicam id_usuario/id_criador do usuário autenticado. A matriz de cobertura no apêndice detalha rota a rota.
5	**Bloqueio de simulado privado na rota de realização**	`rotas/simulado.py:145-147`	fazer_simulado compara eh_privado e id_criador com current_user.id antes de carregar questões. O padrão correto existe, mas não foi replicado em corrigir, ranking e PDF (F-04 a F-06).
6	**Fail-fast para segredos obrigatórios**	`app.py:64-68`; `banco_dados_aluno.py:24-27`	A aplicação encerra se FLASK_SECRET_KEY, SUPABASE_URL, SUPABASE_ANON_KEY ou APP_TOKEN faltarem. Não há fallback do tipo ${VAR:-valor-default}.
7	**Nenhum segredo real commitado no histórico git**	`git log --all -- .env`; `git ls-files`	.env e desktop/token.local.js nunca foram rastreados; o único sk-proj- do histórico é o placeholder 'sk-proj-sua-chave-openai-aqui' em .env.example. A service_role não aparece no repositório.
8	**Jinja2 com autoescape e site público sem sink DOM**	`templates/*.html`; `site/index.html:214-279`	A maior parte das saídas usa {{ }} com autoescape. O site de confirmação usa textContent para mensagens de erro, sem innerHTML com dados do usuário.
9	**Hardening do Electron**	`desktop/main.js:135-139`	contextIsolation: true, nodeIntegration: false e sandbox: true reduzem o impacto de um XSS no renderer, embora não impeçam ações same-origin.
10	**Mensagens de autenticação genéricas e senha mínima**	`rotas/auth.py:17,150-153,223-225`	Login inválido não revela se o e-mail existe; reset de senha usa mensagem genérica; a senha mínima é 8 caracteres.
11	**Superfície de rede restrita ao loopback**	`app.py:332`	O servidor de desenvolvimento sobe em 127.0.0.1:5000; não há Docker/CI/Helm/Terraform no repositório que exponha a aplicação por padrão.
12	**Escrita do aluno exige token + identidade do dono (documentado)**	`docs/banco_dados.md:29,36`	As policies de escrita descritas exigem eh_app_aluno() e eh_dono_do_aluno(...); mesmo com anon key + APP_TOKEN, a escrita em nome de outro aluno é bloqueada. Evidência documental, não SQL executado.

Observação: a escrita do aluno exige *x-app-token* e identidade do dono segundo a documentação (evidência documental, não SQL executado). O CSRF é global e não há *csrf.exempt* no projeto; a única ressalva é que o token CSRF também fica legível por um XSS same-origin, o que reforça a prioridade de F-11.

5. Pontos fracos e risco sistêmico

Os pontos abaixo explicam por que os achados não são incidentes isolados: eles compartilham causa e se reforçam. A ordem reflete o risco percebido, não a ordem numérica dos achados.

CRÍTICA **O SELECT é a porteira aberta**

A privacidade entre alunos não é garantida em SELECT. A anon key pública transforma a leitura em um dump cross-tenant sem autenticação. É o risco sistêmico que prevalece sobre os demais.

ALTA **IDORs de aplicação sobrevivem ao RLS**

Cadernos e simulados privados têm consultas por ID sem dono. Mesmo que o RLS de SELECT seja corrigido, essas rotas continuam vazando ou aceitando escrita cross-tenant.

ALTA **XSS armazenado em comentários**

Texto e nome de usuário entram crus no innerHTML. O CSP com unsafe-inline permite o onerror. O payload atinge qualquer leitor da questão, não só o autor.

MÉDIA **Placar confiado ao cliente na rota legada**

/api/responder aceita is_correct do JSON. A rota principal está correta, mas a alternativa permite forjar histórico e estatísticas.

BAIXA **APP_TOKEN não é uma fronteira real**

O token é compartilhado e extraível do app.asar. Ele fecha o curl fácil, mas não identifica usuário; a barreira real precisa ser o JWT + owner no banco.

INFORMATIVA **Documentação promete mais do que o código entrega**

docs/rotas/caderno.md:111-112 e docs/rotas/simulado.py.md:100,152 afirmam verificação compulsória de dono em todas as rotas; o código contradiz em cadernos e em três rotas de simulado.

6. Tabela de achados

Os 15 achados confirmados estão ordenados por severidade e depois por categoria. A coluna *Local* aponta o arquivo e as linhas exatas; o detalhamento na seção 7 traz trecho, motivo de exploração e correção sugerida. Não há achado especulativo nesta tabela.

ID	Severidade	Cat.	Título	Local
F-01	CRÍTICA	1	RLS de SELECT aberta em 13 tabelas de dados do aluno	`docs/banco_dados.md:27,38`; `desktop/main.js:26-27`; `site/index.html:194-195`
F-02	ALTA	3	IDOR na visualização de caderno	`rotas/caderno.py:29-39`
F-03	ALTA	3	Inserção cross-tenant em caderno alheio	`rotas/caderno.py:114-125,37,168-174`
F-04	ALTA	3	Correção de simulado privado sem verificação de dono	`rotas/simulado.py:186-220,224-230`
F-06	ALTA	3	PDF de simulado privado expõe questões e gabarito	`rotas/simulado.py:310-319,351-360`
F-11	ALTA	5	XSS armazenado em comentários (texto e nome de usuário)	`rotas/questao.py:326-354,311-320`; `static/js/questoes_common.js:229-238`; `app.py:146-151`; `rotas/conta.py:35-44`
F-07	MÉDIA	1	Endpoint público de comentários retorna id_usuario	`rotas/questao.py:311-320,350`
F-08	MÉDIA	2	/api/responder confia no is_correct enviado pelo cliente	`rotas/questao.py:287-301`
F-05	MÉDIA	3	Ranking de simulado privado exposto	`rotas/simulado.py:278-296`
F-09	BAIXA	3	pular_topico_edital sem verificação de dono	`rotas/edital.py:578-583,157-160`
F-10	BAIXA	3	criar_card aceita id_categoria de outro usuário	`rotas/flashcards.py:114-133,50-54`
F-14	BAIXA	4	APP_TOKEN embarcado no cliente não é fronteira de autorização	`desktop/main.js:32-38`; `desktop/token.local.js:3`; `desktop/dist/win-unpacked/resources/app.asar`
F-12	BAIXA	5	Markdown e |safe sem sanitização em múltiplas superfícies	`static/js/questoes_common.js:98-105`; `static/js/aluno_estudar_edital.js:171-174`; `templates/resumos/ver_materia.html:147`; `templates/flashcards/estudar.html:116-117`; `templates/flashcards/gerenciar.html:35,38`; `templates/components/card_questao.html:20`
F-13	INFORMATIVA	4	Anon key e URL do Supabase hardcoded no cliente	`desktop/main.js:26-27`; `site/index.html:194-195`
F-15	INFORMATIVA	4	Segredos locais em texto puro no .env	`.env:4,13,16`

Legenda de categorias: 1 — isolamento de inquilino/dono; 2 — permissão definida no navegador; 3 — IDOR; 4 — chaves expostas; 5 — inputs sem tratamento (XSS). **A categoria 2 não se aplica à UI administrativa** — não existe tela/rota de admin neste repositório e o papel é fixado em *aluno* no servidor; o único achado é de lógica de negócio confiada ao cliente.

7. Detalhamento dos achados

Cada bloco traz a localização exata, a descrição verificada, o motivo de exploração, o impacto e a correção sugerida. Onde a evidência é documental, a condição está explícita. Trechos de código e nomes de tabelas aparecem em fonte monoespaçada.

CRÍTICA F-01 — RLS de SELECT aberta em 13 tabelas de dados do aluno

Local: `docs/banco_dados.md:27,38; desktop/main.js:26-27; site/index.html:194-195`

```
# docs/banco_dados.md:27
"Dados do aluno (16 tabelas: usuarios, historico_respostas,
 comentarios_questao, flashcards, ...): SELECT livre ..."

# docs/banco_dados.md:38
"a leitura dos dados de estudo é liberada pelo desenho atual
 (privacidade entre alunos não é garantida em SELECT, com
 exceção das 3 tabelas de edital)"
```

Descrição. A documentação do projeto declara SELECT livre para o papel anon em usuarios, historico_respostas, comentarios_questao, flashcards, flashcard_categorias, resumos_estudo, cadernos, caderno_questoes, filtros_salvos, simulados, simulados_questoes e simulados_resultados (além de editais_*). A anon key é pública e está embutida no cliente. Com ela, qualquer pessoa pode chamar o PostgREST e ler os dados de todos os alunos, sem login.

Por que é explorável / impacto. Exposição cross-tenant de e-mail/nome, histórico de respostas, comentários, flashcards, resumos, cadernos, filtros, simulados e resultados; base para phishing e para os IDORs encontrados.

Correção sugerida. Recriar as policies de SELECT como owner-only (auth.uid() = id_usuario, ou auth_id em usuarios); separar conteúdo público de dados de aluno; versionar as migrations de RLS neste repositório e testar com dois usuários e com a anon key sem JWT.

Condição de evidência. *Evidência documental: as migrations de RLS não estão no repositório. A exploração depende de a configuração descrita em docs/banco_dados.md estar vigente na produção. Nenhuma consulta real foi executada.*

ALTA F-02 — IDOR na visualização de caderno

Local: `rotas/caderno.py:29-39`

```
# rotas/caderno.py:33
query_nome = supabase.table("cadernos").select("nome_caderno").eq("id_caderno", id_caderno).single()

# rotas/caderno.py:37
query_ids = supabase.table("caderno_questoes").select("id_questao").eq("id_caderno", id_caderno)

# nenhum filtro por current_user.id / id_usuario
```

Descrição. visualizar_caderno busca o nome do caderno por id_caderno (linha 33) e as questões por id_caderno (linha 37), sem filtrar por current_user.id. Também usa session.get('id_usuario') (linha 31) para o histórico. Qualquer aluno autenticado pode enumerar /aluno/caderno/<id> e ler cadernos alheios.

Por que é explorável / impacto. Leitura de conteúdo de estudo alheio; IDs numéricos são triviais de enumerar.

Correção sugerida. Adicionar .eq('id_usuario', current_user.id) nas duas consultas, usar current_user.id no histórico e devolver 404 quando o caderno não for do usuário.

ALTA F-03 — Inserção cross-tenant em caderno alheio

Local: `rotas/caderno.py:114-125,37,168-174`

```
# rotas/caderno.py:116
id_caderno = dados.get('id_caderno')

# rotas/caderno.py:119-123
supabase.table("caderno_questoes").insert({
    "id_caderno": id_caderno,
    "id_questao": id_questao,
    "id_usuario": current_user.id
})
```

Descrição. salvar_na_pasta aceita id_caderno do corpo da requisição (linha 116) e insere em caderno_questoes com id_usuario=current_user.id (linha 122), sem validar se o caderno pertence ao usuário. A vítima vê a questão porque visualizar_caderno filtra só por id_caderno (linha 37); não consegue removê-la porque remover_questao_caderno filtra pelo id_usuario do chamador (linhas 168-174).

Por que é explorável / impacto. Injeção/poluição persistente de cadernos alheios, de difícil remoção pela vítima.

Correção sugerida. Validar a posse do caderno antes do insert; reforçar no banco com FK/constraint ou policy que amarre caderno_questoes.id_usuario ao dono de cadernos.id_usuario.

ALTA F-04 — Correção de simulado privado sem verificação de dono

Local: `rotas/simulado.py:186-220,224-230`

```
# rotas/simulado.py:188
res_simulado = safe_execute(
    supabase.table('simulados').select('*').eq('id', id_simulado).single())
# ^ nenhum check de eh_privado / id_criador

# rotas/simulado.py:215-220
detalhes_resultado.append({
    'id_questao': q_id,
    'resposta_aluno': resposta_aluno,
    'resposta_correta': correta,
    'correto': eh_correto
})
```

Descrição. corrigir_simulado carrega o simulado por ID (linha 188), calcula o gabarito (194-201) e devolve resposta_correta no resultado (215-220), sem repetir a checagem de eh_privado/id_criador existente em fazer_simulado (145-147). Qualquer aluno autenticado pode dar POST em /aluno/simulados/corrigir/<id> de um simulado privado alheio e obter o gabarito; o insert em simulados_resultados (224-230) também é aceito.

Por que é explorável / impacto. Quebra do sigilo do simulado privado, exposição do gabarito e ranking poluído.

Correção sugerida. Extrair um guard único (_simulado_do_usuario_ou_404) e chamá-lo em corrigir, ranking e PDF antes de qualquer consulta de questões/resultados.

ALTA F-06 — PDF de simulado privado expõe questões e gabarito

Local: `rotas/simulado.py:310-319,351-360`

```
# rotas/simulado.py:312-314
res_simulado = safe_execute(
    supabase.table('simulados').select('*').eq('id', id_simulado).single()
)
# ^ nenhum check de eh_privado / id_criador

# rotas/simulado.py:360
q['letra_gabarito'] = letra_certa if letra_certa else 'N/A'
```

Descrição. baixar_pdf_simulado busca o simulado por ID (312-314) e monta a prévia com questões/alternativas (321-360), incluindo letra_gabarito (351-360), sem checar eh_privado/id_criador. Qualquer aluno autenticado pode acessar /simulado/<id>/pdf de um simulado privado alheio e obter a prova e o gabarito.

Por que é explorável / impacto. Vazamento de conteúdo e gabarito de simulados privados.

Correção sugerida. Aplicar o guard de dono antes de buscar questões e alternativas; testar não-dono.

ALTA F-11 — XSS armazenado em comentários (texto e nome de usuário)

Local: `rotas/questao.py:326-354,311-320; static/js/questoes_common.js:229-238; app.py:146-151; rotas/conta.py:35-44`

```
// static/js/questoes_common.js:229-238
html += `
  <div class="mb-2 p-2 bg-white rounded border shadow-sm text-dark">
    <strong class="small text-primary">... ${comentario.nome_usuario || 'Aluno'}</strong>
    <p class="mb-0 small">${comentario.texto_comentario || comentario.texto || ''}</p>
  </div>`;
...
divLista.innerHTML = html;
```

Descrição. salvar_comentario grava texto e nome_usuario sem sanitização (348-353). O nome_usuario vem de usuarios.username, editável em /aluno/conta/ (conta.py:35-44). A listagem pública busca os comentários (311-320) e o JS monta HTML com template literals e innerHTML (229-238), interpolando nome_usuario (232) e texto_comentario (235). Não há DOMPurify/bleach/nh3 no projeto; o CSP permite 'unsafe-inline' em script-src (app.py:146-151), então <img src=x onerror=...> executa. O payload é armazenado e disparado para qualquer usuário que expanda os comentários da questão.

Por que é explorável / impacto. Execução de JS no contexto do app; ações como a vítima, leitura de dados e exfiltração. Em Electron, contextIsolation/sandbox limitam o acesso ao Node, mas não ao mesmo-origin.

Correção sugerida. Sanitizar na saída com DOMPurify (allowlist) e/ou no servidor com bleach/nh3; renderizar nome/texto com textContent; remover 'unsafe-inline' do CSP usando nonces/hashes; adicionar object-src 'none' e base-uri 'self'.

MÉDIA F-07 — Endpoint público de comentários retorna id_usuario

Local: `rotas/questao.py:311-320,350`

```
# rotas/questao.py:314-320
query_comentarios = supabase.table("comentarios_questao") \
    .select("*") \
    .eq("id_questao", id_questao) \
    .order("criado_em", desc=True)
res = safe_execute(query_comentarios)
return jsonify(res.data if res.data else [])
```

Descrição. GET /api/comentarios/<id_questao> não exige autenticação, usa select('*') (314-316) e devolve todas as colunas, incluindo id_usuario (350). Isso entrega UUIDs de alunos a qualquer visitante e serve de pivô para as tabelas com SELECT aberto.

Por que é explorável / impacto. Exposição de identificadores e facilitação de correlação com usuarios e outros dados.

Correção sugerida. Selecionar colunas explícitas (id_comentario, nome_usuario, texto_comentario, criado_em), nunca retornar id_usuario, limitar/paginar e aplicar rate limit.

MÉDIA F-08 — /api/responder confia no is_correct enviado pelo cliente

Local: `rotas/questao.py:287-301`

```
# rotas/questao.py:294
is_correct = data.get('is_correct')

# rotas/questao.py:296-301
query_historico = supabase.table("historico_respostas").insert({
    "id_usuario": current_user.id,
    "id_questao": id_questao,
    "id_alternativa_marcada": id_alternativa,
    "is_correct": is_correct
})
```

Descrição. O endpoint lê is_correct do JSON (linha 294) e grava diretamente em historico_respostas (296-301). O cliente pode enviar true para qualquer resposta, inflando acertos, ofensiva e estatísticas. A rota principal /aluno/responder calcula a correção no servidor (33-47), mas essa rota alternativa permanece e não é chamada pelo frontend atual.

Por que é explorável / impacto. Integridade de progresso/estatísticas; possível manipulação de indicadores derivados.

Correção sugerida. Ignorar is_correct do cliente e calcular no servidor como /aluno/responder; se a rota for legada, removê-la.

MÉDIA F-05 — Ranking de simulado privado exposto

Local: `rotas/simulado.py:278-296`

```
# rotas/simulado.py:280-282
res_simulado = safe_execute(
    supabase.table('simulados').select('titulo, eh_privado').eq('id', id_simulado).single()
)
# ^ busca eh_privado, mas nao compara com current_user

# rotas/simulado.py:289-295
res_ranking = safe_execute(
    supabase.table('simulados_resultados')
    .select('nota, respostas_corretas, total_questoes, data_conclusao, usuarios(username)')
    .eq('id_simulado', id_simulado)
    .order('nota', desc=True)
    .limit(50))
```

Descrição. ver_ranking_simulado busca titulo, eh_privado (280-282), mas não compara eh_privado/id_criador com current_user; em seguida lista resultados e usernames (289-296). Qualquer aluno autenticado com o ID vê o ranking de um simulado privado alheio.

Por que é explorável / impacto. Exposição de participantes, notas e desempenho de um simulado privado.

Correção sugerida. Aplicar o mesmo guard de dono; para simulado privado, permitir ranking apenas ao criador ou desabilitar a tela.

BAIXA F-09 — pular_topico_edital sem verificação de dono

Local: `rotas/edital.py:578-583,157-160`

```
# rotas/edital.py:581-583
def pular_topico_edital(id_edital):
    success, msg, _ = _avancar_ciclo_edital(id_edital, marcar_concluido=True)

# rotas/edital.py:157-160: _avancar_ciclo_edital busca o progresso
res_prog = safe_execute(
    supabase.table("editais_progresso").select("*").eq("id_edital", id_edital).single())
```

Descrição. A rota recebe id_edital (578-583) e chama _avancar_ciclo_edital sem checar editais_ciclo.id_usuario. A função consulta editais_progresso só por id_edital (157-160). Hoje o RLS owner-only das tabelas de progresso (docs:29) impede o cross-tenant, mas a ausência de check é uma regressão latente.

Por que é explorável / impacto. Se o RLS for afrouxado/alterado, um aluno avança o ciclo de outro.

Correção sugerida. Adicionar checagem de posse como em responder_edital (485-490) antes de chamar a função.

BAIXA F-10 — criar_card aceita id_categoria de outro usuário

Local: rotas/flashcards.py:114-133,50-54

```
# rotas/flashcards.py:118
id_cat = dados.get('id_categoria')

# rotas/flashcards.py:127-133
query = supabase.table("flashcards").insert({
    "id_usuario": user_id_str,
    "id_categoria": id_cat,
    "frente": frente,
    "verso": verso,
    "proxima_revisao": datetime.now().isoformat()
})
```

Descrição. A rota lê id_categoria do JSON (118) e insere o flashcard com id_usuario=current_user.id (127-133), sem validar se a categoria pertence ao usuário. O card não aparece na lista da vítima (filtro por id_usuario em 52-53), mas cria referência órfã/cross-tenant e permite poluir categorias alheias.

Por que é explorável / impacto. Baixo; integridade referencial e possível confusão de dados.

Correção sugerida. Validar a posse da categoria antes do insert; reforçar com FK/constraint/RLS.

BAIXA F-14 — APP_TOKEN embarcado no cliente não é fronteira de autorização

Local: desktop/main.js:32-38; desktop/token.local.js:3; desktop/dist/win-unpacked/resources/app.asar

```
// desktop/main.js:32-38
let APP_TOKEN;
try {
  ({ APP_TOKEN } = require('./token.local.js'));
} catch (e) {
  console.error('[shell] ERRO: desktop/token.local.js ausente...');
  process.exit(1);
}
// O token foi localizado no bundle: desktop/dist/win-unpacked/resources/app.asar
```

Descrição. O shell carrega desktop/token.local.js; o token está na linha 3 e foi encontrado no bundle app.asar. A documentação reconhece que ele é extraível (docs/banco_dados.md:36). Como é compartilhado por todos os clientes, não identifica usuário e não deve ser a única barreira de escrita.

Por que é explorável / impacto. Baixa; permite replicar escritas do próprio usuário, mas não escrever como outro.

Correção sugerida. Tratar como identificador de app, não segredo; manter auth.uid() + owner como barreira; rotacionar; considerar attestation/assinatura.

BAIXA F-12 — Markdown e |safe sem sanitização em múltiplas superfícies

Local: static/js/questoes_common.js:98-105; static/js/aluno_estudar_edital.js:171-174; templates/resumos/ver_materia.html:147; templates/flashcards/estudar.html:116-117; templates/flashcards/gerenciar.html:35,38; templates/components/card_questao.html:20

```
// static/js/questoes_common.js:98,105
let htmlRaw = marked.parse(textoProtegido, { breaks: true, gfm: true });
...
divTexto.innerHTML = htmlRaw;

<!-- templates/flashcards/gerenciar.html:35,38 -->
<div class="markdown-text">{{ card.frente|safe }}</div>
<div class="markdown-text">{{ card.verso|safe }}</div>
```

Descrição. O projeto renderiza Markdown em HTML e injeta com innerHTML sem sanitização em explicações de IA, editais, resumos e flashcards; também usa |safe em flashcards e no enunciado da questão. Não há biblioteca de sanitização nas dependências. Hoje o conteúdo é majoritariamente gerado por IA, administrativo ou privado do próprio aluno, então não foi confirmado vetor cross-user; é falha de defesa em profundidade que se torna explorável se esse conteúdo passar a ser compartilhado/importado.

Por que é explorável / impacto. Latente; XSS se conteúdo não confiável chegar a essas superfícies.

Correção sugerida. Sanitizar todo HTML de saída; preferir textContent; allowlist de Markdown; adicionar DOMPurify/bleach e testes de payload.

INFORMATIVA F-13 — Anon key e URL do Supabase hardcoded no cliente

Local: `desktop/main.js:26-27; site/index.html:194-195`

```
// desktop/main.js:26-27
const SUPABASE_URL = 'https://ivhuaevrrkrnagbqmnwc.supabase.co';
const SUPABASE_ANON_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';

<!-- site/index.html:194-195 -->
var SUPABASE_URL = 'https://ivhuaevrrkrnagbqmnwc.supabase.co';
var SUPABASE_ANON_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';
```

Descrição. A URL e a anon key do Supabase estão embutidas no shell Electron e na página pública. A anon key é pública por design (JWT role=anon) e não deve ser tratada como segredo; o risco real é a policy de SELECT aberta (F-01). Não há service_role no repositório.

Por que é explorável / impacto. Informativa; é o habilitador de F-01, não um segredo vazado.

Correção sugerida. Manter pública se o RLS for corrigido; documentar que não é segredo; monitorar uso e não rotacionar como se fosse credencial privilegiada.

INFORMATIVA F-15 — Segredos locais em texto puro no .env

Local: `.env:4,13,16`

```
# .env:4,13,16 (valores reais; arquivo gitignored)
FLASK_SECRET_KEY=<64 hex>
APP_TOKEN=<64 hex>
OPENAI_API_KEY=sk-proj-...
```

Descrição. .env contém FLASK_SECRET_KEY (linha 4), APP_TOKEN (13) e OPENAI_API_KEY (16) com valores reais. O arquivo é gitignored e nunca foi commitado (verificado com git log --all -- .env e git ls-files), então não é vazamento de repositório. É higiene: a chave OpenAI parece não ser usada pelo fluxo BYOK e deve ser removida/rotacionada; a FLASK_SECRET_KEY deve ser rotacionada se o arquivo foi compartilhado.

Por que é explorável / impacto. Informativa; exposição local/backup, não exposição no git.

Correção sugerida. Remover a OPENAI_API_KEY não usada; rotacionar; usar cofre/DPAPI; adicionar gitleaks no pre-commit.

8. Recomendações priorizadas

P1 fecha exposição ativa ou integridade de dados e deve entrar antes do próximo build distribuído. **P2** reduz superfície e corrige defesa em profundidade. **P3** é sistêmico: testes, análise estática e governança de RLS para evitar regressão.

Prioridade	Ação	Como executar	Achados
P1	**Fechar o SELECT cross-tenant no Supabase**	Recriar as policies de SELECT das tabelas de aluno como owner-only; separar conteúdo público; versionar as migrations; validar com anon key e com dois usuários.	`F-01, F-07`
P1	**Corrigir os IDORs de cadernos**	Adicionar id_usuario em visualizar_caderno e validar a posse do caderno em salvar_na_pasta; reforçar no banco.	`F-02, F-03`
P1	**Proteger simulados privados nas três rotas**	Aplicar o mesmo guard de fazer_simulado em corrigir, ranking e PDF; devolver 404 para não-dono antes de qualquer consulta.	`F-04, F-05, F-06`
P1	**Eliminar o XSS armazenado de comentários**	Sanitizar na saída (DOMPurify/bleach), usar textContent para nome/texto e remover unsafe-inline do CSP com nonces/hashes.	`F-11`
P1	**Remover ou corrigir /api/responder**	Calcular is_correct no servidor ou apagar a rota legada; adicionar teste que envia true para uma resposta errada.	`F-08`
P2	**Reduzir a exposição do endpoint de comentários**	Selecionar colunas explícitas, não devolver id_usuario, paginar e limitar a taxa.	`F-07`
P2	**Fechar checks de posse em editais e flashcards**	Adicionar o guard de dono em pular_topico_edital e validar id_categoria em criar_card; teste com dois usuários.	`F-09, F-10`
P2	**Sanitizar Markdown e remover |safe de conteúdo dinâmico**	Aplicar allowlist de Markdown, DOMPurify/bleach e testes de payload em explicações, resumos, flashcards e enunciados.	`F-12`
P2	**Reposicionar o APP_TOKEN e limpar segredos locais**	Tratar o token como identificador de app; manter JWT + owner como barreira; remover a OPENAI_API_KEY não usada e rotacionar o que foi compartilhado.	`F-14, F-15`
P2	**Adicionar verificação de segredos no fluxo de desenvolvimento**	gitleaks no pre-commit/CI, pip-audit/npm audit e revisão do .env.example.	`F-13, F-14, F-15`
P3	**Criar testes de regressão cross-tenant**	Dois usuários de teste: B não lê/edita A em cadernos, simulados, editais, flashcards e resumos; anon não lê dados de aluno.	`F-01 a F-10`
P3	**Adicionar análise estática e SCA no CI**	bandit/semgrep para Python, eslint-plugin-security para JS, pip-audit e npm audit; falhar o build em achados altos.	`Transversal`
P3	**Versionar e testar as policies RLS**	Migrations no repo, script de teste com anon/authenticated e documentação de exceções de leitura pública.	`F-01`
P3	**Auditar logs e dados sensíveis**	Não logar session/refresh token; revisar logs de login e de queries; mascarar e-mail/UUID em telemetria.	`Transversal`

Sequência sugerida: (1) fechar o SELECT e os IDORs; (2) eliminar o XSS e a rota legada; (3) reduzir exposição e sanear sinks; (4) institucionalizar testes e verificação de segredos. As issues da seção 10 já estão no formato de execução.

9. Apêndice — Matriz de cobertura de rotas

Foram inventariadas 35 rotas que recebem identificador, mutam dados ou lidam com credenciais. A coluna *Controle* registra o filtro efetivamente presente no código, não a intenção documentada. **OK** significa que o dono é derivado da sessão; **IDOR** significa que o objeto é buscado apenas pelo ID recebido.

Rota	Arquivo:linha	Controle verificado	Status
`GET /aluno/caderno/<id_caderno>`	`rotas/caderno.py:26`	Nenhum (só id_caderno)	IDOR
`POST /api/cadernos/salvar`	`rotas/caderno.py:111`	Nenhum no id_caderno	IDOR
`POST /api/cadernos/excluir/<id_caderno>`	`rotas/caderno.py:134`	id_usuario	OK
`POST /api/cadernos/remover_questao`	`rotas/caderno.py:150`	id_usuario	OK
`GET /aluno/simulados/fazer/<id>`	`rotas/simulado.py:132`	eh_privado + id_criador	OK
`POST /aluno/simulados/corrigir/<id>`	`rotas/simulado.py:183`	Nenhum	IDOR
`POST /aluno/simulados/excluir/<id>`	`rotas/simulado.py:256`	id_criador	OK
`GET /aluno/simulados/ranking/<id>`	`rotas/simulado.py:275`	Nenhum	IDOR
`GET /simulado/<id>/pdf`	`rotas/simulado.py:307`	Nenhum	IDOR
`GET /aluno/editais/estudar/<id>`	`rotas/edital.py:379`	id_usuario	OK
`POST /aluno/editais/responder`	`rotas/edital.py:476`	id_usuario	OK
`POST /aluno/editais/pular/<id>`	`rotas/edital.py:578`	Nenhum (RLS mitiga)	RISCO LATENTE
`GET /aluno/editais/dashboard/<id>`	`rotas/edital.py:598`	id_usuario	OK
`POST /aluno/editais/excluir/<id>`	`rotas/edital.py:712`	id_usuario	OK
`GET /aluno/flashcards/categoria/<id>`	`rotas/flashcards.py:38`	id_usuario	OK
`POST /api/flashcards/criar`	`rotas/flashcards.py:114`	id_categoria não validado	BAIXO
`POST /api/flashcards/revisar`	`rotas/flashcards.py:140`	id_usuario	OK
`POST /api/flashcards/excluir/<id>`	`rotas/flashcards.py:213`	id_usuario	OK
`POST /api/flashcards/categoria/excluir/<id>`	`rotas/flashcards.py:223`	id_usuario	OK
`GET /aluno/resumos/materia/<id>`	`rotas/resumos.py:31`	id_usuario	OK
`POST /api/resumos/excluir/<id>`	`rotas/resumos.py:46`	id_usuario	OK
`POST /api/resumos/excluir-materia/<id>`	`rotas/resumos.py:56`	id_usuario	OK
`POST /aluno/excluir_filtro/<id>`	`rotas/filtro.py:36`	id_usuario	OK
`POST /aluno/api/set-meta`	`rotas/aluno.py:171`	current_user.id	OK
`POST /api/usuario/preferencias`	`rotas/api.py:106`	current_user.id	OK
`POST /aluno/conta/`	`rotas/conta.py:31`	current_user.id	OK
`POST /aluno/conta/resetar-dados`	`rotas/conta.py:110`	current_user.id	OK
`POST /aluno/conta/excluir`	`rotas/conta.py:158`	current_user.id	OK
`POST /aluno/responder`	`rotas/questao.py:24`	current_user.id + score server-side	OK
`POST /api/responder`	`rotas/questao.py:287`	current_user.id, mas is_correct do cliente	FALHA DE INTEGRIDADE
`GET /api/comentarios/<id>`	`rotas/questao.py:311`	Público; devolve id_usuario	EXPOSIÇÃO
`POST /api/comentarios`	`rotas/questao.py:326`	current_user.id	OK (MAS XSS)
`POST /api/comentarios/<id>/reportar`	`rotas/questao.py:363`	current_user.id	OK
`POST /api/ia/config`	`rotas/conta.py:66`	current_user.id	OK
`POST /api/ia/config/remover`	`rotas/conta.py:102`	current_user.id	OK

Status **Risco latente**: o RLS owner-only de *editais_progresso* mitiga hoje, mas não há check de aplicação. **Exposição**: a rota é pública e devolve coluna sensível. **Falha de integridade**: o cliente influencia o placar. **Baixo**: referência cross-tenant sem impacto confirmado na listagem.

10. ISSUES PARA O GITHUB

Os blocos abaixo estão completos e prontos para copiar e colar no GitHub. Cada issue começa com o título em Markdown, seguido de descrição, evidência, exploração, impacto, correção e critérios de aceite. O título do bloco e as labels sugeridas aparecem na faixa acima do texto; o conteúdo entre **--- ISSUE n ---** e **--- FIM ISSUE n ---** é o Markdown integral.

Como usar: crie uma issue por bloco, cole o Markdown integral e aplique as labels sugeridas. A ordem segue a prioridade: crítica, altas, médias, baixas e hardening. Nove issues cobrem os quinze achados.

--- ISSUE 1 ---

Issue 1 — [Segurança] RLS de SELECT aberta expõe dados de todos os alunos
Labels sugeridas: security, severity:critical, database, rls

````
# [Segurança] RLS de SELECT aberta expõe dados de todos os alunos

**Labels sugeridas:** `security`, `severity:critical`, `database`, `rls`

## Descrição do problema

A documentação do projeto declara que as tabelas de dados do aluno têm
`SELECT` livre para o papel `anon`, com exceção apenas das três tabelas de
progresso de edital. A anon key é pública e está embutida no cliente
(`desktop/main.js` e `site/index.html`). Com essa chave, qualquer pessoa
consegue consultar o PostgREST e ler dados de todos os alunos, sem login.

Tabelas citadas na documentação: `usuarios`, `historico_respostas`,
`comentarios_questao`, `flashcards`, `flashcard_categorias`,
`resumos_estudo`, `cadernos`, `caderno_questoes`, `filtros_salvos`,
`simulados`, `simulados_questoes`, `simulados_resultados` e `editais_*`.

## Evidência

- `docs/banco_dados.md:27` — "Dados do aluno (16 tabelas...): SELECT livre".
- `docs/banco_dados.md:38` — "a leitura dos dados de estudo é liberada pelo
  desenho atual (privacidade entre alunos não é garantida em SELECT, com
  exceção das 3 tabelas de edital)".
- `desktop/main.js:26-27` — URL e anon key hardcoded.
- `site/index.html:194-195` — URL e anon key hardcoded.

```http
GET https://<projeto>.supabase.co/rest/v1/usuarios?select=*
apikey: <SUPABASE_ANON_KEY>
```

## Por que é explorável

A policy de SELECT para `anon` é o único controle. A chave é pública por
design, então não há barreira de autenticação. O atacante não precisa de
conta nem de engenharia social: basta extrair a chave do bundle ou do site
e chamar a API REST.

> Observação: não foi executada consulta contra a produção. A exploração
> depende de a configuração documentada estar vigente no projeto Supabase.

## Impacto

- Leitura de e-mail/nome de usuário, histórico de respostas, comentários,
  flashcards, resumos, cadernos, filtros, simulados e resultados.
- Correlação entre alunos e reconstrução de rotina de estudos.
- Base para phishing e para os IDORs descritos em outras issues.

## Sugestão de correção
1. Recriar as policies de SELECT das tabelas de aluno como owner-only:
````

```
   `auth.uid() = id_usuario` (ou `auth_id` em `usuarios`).
2. Manter conteúdo público (questões, alternativas, taxonomia) em tabelas
   separadas ou views públicas, sem misturar com dados de aluno.
3. Validar com dois usuários reais e com a anon key sem JWT.
4. Versionar as migrations de RLS neste repositório.

## Critérios de aceite

- [ ] A anon key sozinha retorna 0 linhas de `usuarios`,
      `historico_respostas`, `comentarios_questao`, `flashcards`,
      `cadernos`, `filtros_salvos` e `simulados`.
- [ ] O usuário B não lê nenhuma linha do usuário A em todas as tabelas
      de dados de aluno.
- [ ] Questões e taxonomia públicas continuam acessíveis.
- [ ] Migrations de RLS estão versionadas e há teste de regressão.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Issue 2 — [Segurança] IDOR em cadernos: leitura e escrita em cadernos de outros alunos

Labels sugeridas: security, severity:high, backend, idor

````
# [Segurança] IDOR em cadernos: leitura e escrita em cadernos de outros alunos

**Labels sugeridas:** `security`, `severity:high`, `backend`, `idor`

## Descrição do problema

Duas rotas de cadernos confiam no `id_caderno` recebido e não verificam se
o caderno pertence ao usuário autenticado:

1. `visualizar_caderno` lê o nome e as questões apenas por `id_caderno`.
2. `salvar_na_pasta` insere uma questão em qualquer `id_caderno`, usando o
   `id_usuario` do atacante. A vítima vê a questão porque a leitura também
   é feita só por `id_caderno`; e não consegue removê-la porque a remoção
   filtra pelo `id_usuario` do chamador.

## Evidência

```python
# rotas/caderno.py:33-37
query_nome = supabase.table("cadernos").select("nome_caderno")\
    .eq("id_caderno", id_caderno).single()
query_ids = supabase.table("caderno_questoes")\
    .select("id_questao").eq("id_caderno", id_caderno)
```

```python
# rotas/caderno.py:119-123
supabase.table("caderno_questoes").insert({
    "id_caderno": id_caderno,
    "id_questao": id_questao,
    "id_usuario": current_user.id
})
```

```python
# rotas/caderno.py:168-174 (remoção)
query_delete = (
    supabase.table("caderno_questoes").delete()
    .eq("id_caderno", id_caderno)
    .eq("id_questao", id_questao_int)
    .eq("id_usuario", user_id)
)
```

## Por que é explorável
````

```
O `id_caderno` é numérico e sequencial. Qualquer aluno autenticado pode
enumerar `/aluno/caderno/1`, `/aluno/caderno/2` etc. e ler o conteúdo; com
um POST em `/api/cadernos/salvar`, pode inserir questões no caderno de
outra pessoa. O RLS de escrita permite o insert porque o `id_usuario`
gravado é o do atacante, não o da vítima.

## Impacto

- Leitura de conteúdo de estudo alheio.
- Poluição persistente de cadernos de terceiros.
- A vítima não consegue remover a questão inserida pelo atacante.

## Sugestão de correção

1. Em `visualizar_caderno`, adicionar `.eq("id_usuario", current_user.id)`
   nas consultas de `cadernos` e `caderno_questoes`; devolver 404 se não
   encontrar.
2. Em `salvar_na_pasta`, buscar o caderno com
   `.eq("id_caderno", id_caderno).eq("id_usuario", current_user.id)` antes
   do insert; rejeitar se não existir.
3. Reforçar no banco: FK/constraint ou policy que amarre
   `caderno_questoes.id_usuario` ao dono de `cadernos.id_usuario`.
4. Adicionar teste com dois usuários.

## Critérios de aceite

- [ ] O usuário B recebe 404/403 ao abrir o caderno do usuário A.
- [ ] O usuário B não consegue inserir questão no caderno do usuário A.
- [ ] O usuário A consegue remover qualquer questão do próprio caderno.
- [ ] Teste de regressão cobre leitura e escrita cross-tenant.
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Issue 3 — [Segurança] Simulados privados acessíveis por ID em correção, ranking e PDF

Labels sugeridas: security, severity:high, backend, idor, simulados

```
# [Segurança] Simulados privados acessíveis por ID em correção, ranking e PDF

**Labels sugeridas:** `security`, `severity:high`, `backend`, `idor`, `simulados`

## Descrição do problema

A rota `fazer_simulado` bloqueia simulado privado de outro usuário, mas o
mesmo guard não foi replicado em três rotas que também aceitam
`id_simulado`:

- `corrigir_simulado`: calcula e devolve o gabarito; também insere um
  resultado para o atacante.
- `ver_ranking_simulado`: lista participantes e notas.
- `baixar_pdf_simulado`: monta a prova e inclui `letra_gabarito`.

Qualquer aluno autenticado com o ID (numérico e sequencial) acessa o
conteúdo de um simulado privado alheio.

## Evidência

```python
# rotas/simulado.py:145-147 — guard correto, só em fazer_simulado
if simulado.get('eh_privado') and simulado.get('id_criador') != current_user.id:
    flash("Você não tem permissão para acessar este simulado.", "danger")
    return redirect('/aluno/simulados')
```

```python
# rotas/simulado.py:188-220 — corrigir sem guard
res_simulado = safe_execute(
```
```

```
    supabase.table('simulados').select('*').eq('id', id_simulado).single())
...
detalhes_resultado.append({
    'id_questao': q_id,
    'resposta_aluno': resposta_aluno,
    'resposta_correta': correta,
    'correto': eh_correto
})
```

```python
# rotas/simulado.py:280-296 — ranking sem guard
res_simulado = safe_execute(
    supabase.table('simulados').select('titulo, eh_privado')
    .eq('id', id_simulado).single())
...
res_ranking = safe_execute(
    supabase.table('simulados_resultados')
    .select('nota, respostas_corretas, total_questoes, data_conclusao, usuarios(username)')
    .eq('id_simulado', id_simulado)
    .order('nota', desc=True).limit(50))
```

```python
# rotas/simulado.py:312-360 — PDF sem guard e com gabarito
res_simulado = safe_execute(
    supabase.table('simulados').select('*').eq('id', id_simulado).single())
...
q['letra_gabarito'] = letra_certa if letra_certa else 'N/A'
```

Por que é explorável

O atacante só precisa estar logado como aluno e conhecer/adivinhar o ID.
Não há verificação de dono nem de privacidade nas três rotas. A correção
ainda grava `simulados_resultados`, permitindo poluir o ranking do simulado
privado da vítima.

Impacto

- Exposição do gabarito e das questões de simulados privados.
- Exposição de participantes, notas e desempenho.
- Poluição de resultados/ranking.
- Quebra da expectativa de privacidade do recurso.

Sugestão de correção

1. Criar um helper único, por exemplo
   `_simulado_do_usuario_ou_404(id_simulado)`, que busca o simulado e
   compara `eh_privado`/`id_criador` com `current_user.id`.
2. Chamar o helper no início de `corrigir_simulado`,
   `ver_ranking_simulado` e `baixar_pdf_simulado`, antes de qualquer
   consulta de questões/resultados.
3. Para simulado privado, permitir ranking apenas ao criador ou desabilitar
   a tela.
4. Testar com dois usuários e com simulado público.

Critérios de aceite

- [ ] Não-dono recebe 403/404 em corrigir, ranking e PDF de simulado privado.
- [ ] Nenhuma questão, alternativa ou gabarito é retornada antes do guard.
- [ ] Criador e simulado público continuam funcionando.
- [ ] Teste de regressão cobre as três rotas.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Issue 4 — [Segurança] XSS armazenado em comentários permite executar JS na sessão da vítima

Labels sugeridas: security, severity:high, frontend, backend, xss

````
# [Segurança] XSS armazenado em comentários permite executar JS na sessão da vítima

**Labels sugeridas:** `security`, `severity:high`, `frontend`, `backend`, `xss`

## Descrição do problema

O backend grava o texto do comentário e o nome de usuário sem sanitização.
O nome vem de `usuarios.username`, que o próprio aluno edita em
`/aluno/conta/`. A listagem pública busca os comentários e o JavaScript
monta o HTML com template literals e `innerHTML`, interpolando o nome e o
texto. Não há DOMPurify, bleach ou nh3 no projeto. O CSP permite
``'unsafe-inline'`` em `script-src`, então um payload com `onerror` executa.

O ataque é armazenado: o payload fica no banco e dispara para qualquer
usuário que abra/expanda os comentários da questão, não apenas para quem o
criou.

## Evidência

```python
# rotas/questao.py:348-353
query_insert = supabase.table("comentarios_questao").insert({
    "id_questao": id_questao,
    "id_usuario": str(current_user.id),
    "nome_usuario": nome_aluno,
    "texto_comentario": texto
})
```

```javascript
// static/js/questoes_common.js:229-238
html += `
<div class="mb-2 p-2 bg-white rounded border shadow-sm text-dark">
  <div class="d-flex justify-content-between align-items-center mb-1">
    <strong class="small text-primary">
      <i class="bi bi-person-circle"></i> ${comentario.nome_usuario || 'Aluno'}
    </strong>
    ...
  </div>
  <p class="mb-0 small">${comentario.texto_comentario || comentario.texto || ''}</p>
</div>`;
...
divLista.innerHTML = html;
```

```python
# app.py:146-151
'script-src': [
    '\'self\'',
    '\'unsafe-inline\'',
    'https://cdn.jsdelivr.net',
    'https://cdn.ckeditor.com',
],
```

```python
# rotas/conta.py:35-44 — username editável e sem escape
novo_username = request.form.get('username')
...
supabase.table("usuarios").update(dados_update).eq("id", current_user.id)
```

Payload mínimo de prova:
```html
````
````

```
<img src=x onerror=alert(document.domain)>
```

Por que é explorável

`innerHTML` não executa `<script>`, mas executa handlers de evento como
`onerror`. O CSP com `'unsafe-inline'` permite o handler. O backend não
escapa nem valida o conteúdo. O endpoint de comentários é público para
leitura e qualquer aluno autenticado pode publicar.

Impacto

- Execução de JavaScript no contexto do app, com a sessão da vítima.
- Ações em nome da vítima (responder, criar/editar dados, denunciar).
- Leitura do DOM/CSRF token e exfiltração de dados.
- No Electron, `contextIsolation`/`sandbox` limitam o acesso ao Node, mas
  não impedem requisições same-origin.

Sugestão de correção

1. Sanitizar a saída com DOMPurify (allowlist) antes de atribuir a
   `innerHTML`; para nome e texto, preferir `textContent`.
2. Sanitizar também no servidor (bleach/nh3) como defesa em profundidade.
3. Remover `'unsafe-inline'` do `script-src` e usar nonces/hashes.
4. Adicionar `object-src 'none'` e `base-uri 'self'` ao CSP.
5. Criar teste de regressão com payloads de XSS.

Critérios de aceite

- [ ] `<img src=x onerror=alert(1)>` é renderizado como texto, sem executar.
- [ ] O nome de usuário também é tratado como texto.
- [ ] O CSP não contém `'unsafe-inline'` em `script-src`.
- [ ] Teste automatizado cobre comentário e username maliciosos.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Issue 5 — [Segurança] /api/responder confia no is_correct enviado pelo cliente

Labels sugeridas: security, severity:medium, backend, integrity

```
# [Segurança] /api/responder confia no is_correct enviado pelo cliente

**Labels sugeridas:** `security`, `severity:medium`, `backend`, `integrity`

## Descrição do problema

O endpoint `/api/responder` lê `is_correct` do JSON enviado pelo cliente e
grava o valor diretamente em `historico_respostas`. O cliente pode enviar
`true` para qualquer resposta, inflando acertos, ofensiva e estatísticas.
A rota principal `/aluno/responder` calcula a correção no servidor, mas a
rota alternativa permanece. O frontend atual chama apenas
`/aluno/responder`, então a rota parece legada.

## Evidência

```python
# rotas/questao.py:287-301
@questao_bp.route('/api/responder', methods=['POST'])
@aluno_required
@limiter.exempt
def salvar_resposta():
    data = request.json
    id_questao = data.get('id_questao')
    id_alternativa = data.get('id_alternativa')
    is_correct = data.get('is_correct')
    query_historico = supabase.table("historico_respostas").insert({
        "id_usuario": current_user.id,
```
```

```
        "id_questao": id_questao,
        "id_alternativa_marcada": id_alternativa,
        "is_correct": is_correct
    })
```

```python
# rotas/questao.py:33-47 — rota principal, correta
query_alt = supabase.table("alternativas_questao")\
    .select("id_alternativa, letra, is_correct")\
    .eq("id_questao", questao_id).order("letra")
...
letra_correta = next((a['letra'] for a in alternativas if a['is_correct']), None)
foi_correta = (letra_escolhida == letra_correta)
```

Por que é explorável

Basta um POST autenticado com `{"id_questao": 1, "id_alternativa": 2, "is_correct": true}`. O servidor não consulta o gabarito nem valida a alternativa; grava o booleano do atacante.

Impacto

- Histórico e estatísticas de estudo falsificados.
- Ofensiva, dashboard e indicadores derivados incorretos.
- Se algum ranking/relatório usar o histórico, pode ser manipulado.

Sugestão de correção

1. Ignorar `is_correct` do cliente.
2. Buscar as alternativas da questão no servidor e calcular `foi_correta = (letra_escolhida == letra_correta)`.
3. Reutilizar a lógica de `/aluno/responder` ou remover a rota legada se não houver chamador.
4. Adicionar teste que envia `is_correct: true` para uma resposta errada.

Critérios de aceite

- [ ] `is_correct` enviado pelo cliente é ignorado.
- [ ] O servidor calcula a correção a partir do gabarito.
- [ ] Teste prova que um payload forjado não altera o histórico.
- [ ] A rota legada foi removida ou documentada como deprecated.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Issue 6 — [Segurança] Endpoint público de comentários expõe id_usuario dos alunos

Labels sugeridas: security, severity:medium, backend, privacy

```
# [Segurança] Endpoint público de comentários expõe id_usuario dos alunos

**Labels sugeridas:** `security`, `severity:medium`, `backend`, `privacy`

## Descrição do problema

`GET /api/comentarios/<id_questao>` não exige autenticação, usa
`select('*')` e devolve todas as colunas da tabela, incluindo `id_usuario`.
Isso entrega UUIDs de alunos a qualquer visitante e serve de pivô para as
tabelas com SELECT aberto (ver issue crítica de RLS).

## Evidência

```python
# rotas/questao.py:311-320
@questao_bp.route('/api/comentarios/<int:id_questao>', methods=['GET'])
@limiter.exempt
```
```

```
def buscar_comentarios(id_questao):
    query_comentarios = supabase.table("comentarios_questao") \
        .select("*") \
        .eq("id_questao", id_questao) \
        .order("criado_em", desc=True)
    try:
        res = safe_execute(query_comentarios)
        return jsonify(res.data if res.data else [])
```

```python
# rotas/questao.py:348-353 — id_usuario é gravado na tabela
supabase.table("comentarios_questao").insert({
    "id_questao": id_questao,
    "id_usuario": str(current_user.id),
    "nome_usuario": nome_aluno,
    "texto_comentario": texto
})
```

Por que é explorável

A rota é pública e não há filtro de colunas. Qualquer pessoa pode chamar `/api/comentarios/1` e receber a lista completa, com UUIDs. Combinado com `usuarios` de SELECT aberto, o UUID vira nome/e-mail.

Impacto

- Exposição de identificadores de alunos.
- Facilita correlação com histórico, cadernos e outros dados.
- Aumenta o impacto da falha de RLS.

Sugestão de correção

1. Selecionar apenas as colunas necessárias:
   `id_comentario, nome_usuario, texto_comentario, criado_em`.
2. Nunca retornar `id_usuario` para o cliente.
3. Paginar/limitar a lista e aplicar rate limit.
4. Se o autor precisar ser identificado para denúncia, usar
   `id_comentario` como referência, não o UUID do usuário.

Critérios de aceite

- [ ] A resposta não contém `id_usuario`.
- [ ] A rota continua funcionando para leitura pública.
- [ ] Há paginação/limite e teste com anon.
- [ ] Teste verifica que o UUID não aparece no JSON.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

Issue 7 — [Segurança] Checks de posse ausentes em editais e flashcards (defesa em profundidade)

Labels sugeridas: security, severity:low, backend, idor

```
# [Segurança] Checks de posse ausentes em editais e flashcards (defesa em profundidade)

**Labels sugeridas:** `security`, `severity:low`, `backend`, `idor`

## Descrição do problema

Duas rotas aceitam IDs de objetos sem validar o dono:

1. `pular_topico_edital` recebe `id_edital` e chama
   `_avancar_ciclo_edital` sem comparar `editais_ciclo.id_usuario` com
   `current_user.id`. Hoje o RLS owner-only das tabelas de progresso
   impede o cross-tenant, mas o código não tem a checagem.
2. `criar_card` aceita `id_categoria` de qualquer usuário. O card é gravado
```

````
   com o `id_usuario` do atacante e não aparece na lista da vítima, mas
   cria referência cross-tenant/órfã.

## Evidência

```python
# rotas/edital.py:578-583
@edital_bp.route('/aluno/editais/pular/<int:id_edital>', methods=['POST'])
@aluno_required
def pular_topico_edital(id_edital):
    success, msg, _ = _avancar_ciclo_edital(id_edital, marcar_concluido=True)
```

```python
# rotas/edital.py:157-160 — consulta só por id_edital
res_prog = safe_execute(
    supabase.table("editais_progresso").select("*")
    .eq("id_edital", id_edital).single())
```

```python
# rotas/flashcards.py:118-133
id_cat = dados.get('id_categoria')
...
supabase.table("flashcards").insert({
    "id_usuario": user_id_str,
    "id_categoria": id_cat,
    "frente": frente,
    "verso": verso,
})
```

## Por que é explorável

Hoje o impacto é limitado pelo RLS: `editais_progresso` é owner-only e o
card do atacante não aparece para a vítima. A falha é de defesa em
profundidade: se uma policy for afrouxada ou uma rota for refatorada, o
check de aplicação não existe para segurar.

## Impacto

- Baixo no estado atual.
- Latente: avanço de ciclo de edital alheio se o RLS falhar.
- Poluição referencial de categorias de flashcards.

## Sugestão de correção

1. Em `pular_topico_edital`, validar a posse do edital como em
   `responder_edital` (linhas 485-490) antes de chamar a função.
2. Em `criar_card`, buscar a categoria com
   `.eq("id_categoria", id_cat).eq("id_usuario", user_id_str)` e rejeitar
   se não existir.
3. Reforçar com FK/constraint/RLS.
4. Testar com dois usuários.

## Critérios de aceite

- [ ] Não-dono recebe 403/404 em `pular_topico_edital`.
- [ ] Não-dono não consegue criar card em categoria alheia.
- [ ] Testes cobrem os dois cenários.
- [ ] Documentação de rotas reflete o comportamento real.
````

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

Issue 8 — [Segurança] Markdown e |safe sem sanitização em múltiplas telas

Labels sugeridas: security, severity:low, frontend, xss, hardening

```
# [Segurança] Markdown e |safe sem sanitização em múltiplas telas

**Labels sugeridas:** `security`, `severity:low`, `frontend`, `xss`, `hardening`

## Descrição do problema

O projeto renderiza Markdown em HTML e injeta com `innerHTML` sem
sanitização em várias telas; também usa `|safe` em conteúdo dinâmico. Não
há biblioteca de sanitização nas dependências. Hoje o conteúdo é
majoritariamente gerado por IA, administrativo ou privado do próprio aluno,
então não foi confirmado vetor cross-user. É uma falha de defesa em
profundidade que se torna explorável se esse conteúdo passar a ser
compartilhado, importado ou influenciado por outro usuário.

## Evidência

```javascript
// static/js/questoes_common.js:98-105
let htmlRaw = marked.parse(textoProtegido, { breaks: true, gfm: true });
...
divTexto.innerHTML = htmlRaw;
```

```javascript
// static/js/aluno_estudar_edital.js:171-174
if (window.marked) {
    resultDiv.innerHTML = marked.parse(data.explicacao);
} else {
    resultDiv.innerHTML = data.explicacao.replace(/\n/g, '<br>');
}
```

```javascript
// templates/resumos/ver_materia.html:147
el.innerHTML = marked.parse(raw, { breaks: true, gfm: true });
```

```html
<!-- templates/flashcards/gerenciar.html:35,38 -->
<div class="text-dark-emphasis mb-2 markdown-text">{{ card.frente|safe }}</div>
<div class="text-muted small markdown-text">{{ card.verso|safe }}</div>

<!-- templates/components/card_questao.html:20 -->
<p class="mb-3">{{ q.enunciado | safe}}</p>
```

Por que é explorável

`marked.parse` não sanitiza HTML; a saída vai direto para `innerHTML`. Se
qualquer uma dessas fontes (IA, admin, importação, compartilhamento) passar
a aceitar conteúdo de outro usuário, o payload executa. A ausência de
sanitizador é a falha estrutural; o vetor concreto depende da evolução do
produto.

Impacto

- Latente: XSS se conteúdo não confiável chegar às telas.
- Aumenta a superfície de ataque de qualquer feature futura de
  compartilhamento/importação.
- Dificulta a revisão de segurança de novos pontos de renderização.

Sugestão de correção

1. Adicionar DOMPurify (frontend) e/ou bleach/nh3 (backend).
```

```
2. Sanitizar toda saída de Markdown antes de `innerHTML`.
3. Preferir `textContent` para conteúdo que não precisa de HTML.
4. Remover `|safe` de conteúdo dinâmico ou sanitizar antes do template.
5. Criar testes de payload para cada tela.

## Critérios de aceite

- [ ] Existe sanitizador nas dependências e ele é aplicado nos pontos
      listados.
- [ ] `|safe` não é usado em conteúdo de usuário/IA sem sanitização.
- [ ] Testes de XSS cobrem explicação, resumo, flashcard e enunciado.
- [ ] CSP sem `'unsafe-inline'` em `script-src`.
```

--- FIM ISSUE 8 ---

--- ISSUE 9 ---

Issue 9 — [Segurança] Segredos embarcados no cliente e higiene do .env

Labels sugeridas: security, severity:low, hardening, secrets

```
# [Segurança] Segredos embarcados no cliente e higiene do .env

**Labels sugeridas:** `security`, `severity:low`, `hardening`, `secrets`

## Descrição do problema

Há três pontos relacionados, com impactos diferentes:

1. A URL e a anon key do Supabase estão hardcoded no Electron e no site.
   A anon key é pública por design; não é um segredo vazado. O risco real é
   a policy de SELECT aberta (issue crítica).
2. O `APP_TOKEN` é carregado de `desktop/token.local.js` e foi encontrado
   no bundle `app.asar`. A documentação reconhece que ele é extraível.
   Como é compartilhado por todos os clientes, não identifica usuário e não
   deve ser a única barreira de escrita.
3. O `.env` local contém `FLASK_SECRET_KEY`, `APP_TOKEN` e uma
   `OPENAI_API_KEY` com valores reais. O arquivo é gitignored e nunca foi
   commitado; é higiene local, não vazamento de repositório.

## Evidência

```javascript
// desktop/main.js:26-27
const SUPABASE_URL = 'https://ivhuaevrrkrnagbqmnwc.supabase.co';
const SUPABASE_ANON_KEY = 'eyJhbGciOi...';
```

```html
<!-- site/index.html:194-195 -->
var SUPABASE_URL = 'https://ivhuaevrrkrnagbqmnwc.supabase.co';
var SUPABASE_ANON_KEY = 'eyJhbGciOi...';
```

```javascript
// desktop/main.js:32-38
let APP_TOKEN;
try {
  ({ APP_TOKEN } = require('./token.local.js'));
} catch (e) {
  console.error('[shell] ERRO: desktop/token.local.js ausente...');
  process.exit(1);
}
```

```text
# .env:4,13,16 (valores reais; arquivo gitignored)
FLASK_SECRET_KEY=<64 hex>
APP_TOKEN=<64 hex>
```
```

````
OPENAI_API_KEY=sk-proj-...
```

## Por que é explorável

A anon key sozinha não dá escrita, mas dá leitura enquanto o RLS estiver
aberto. O `APP_TOKEN` é extraível do app.asar e compartilhado; quem o
extrair consegue replicar escritas do próprio usuário, mas não escrever
como outro aluno, porque o RLS também exige o JWT/dono. A `OPENAI_API_KEY`
local parece não ser usada pelo fluxo BYOK e deve ser removida.

## Impacto

- Anon key: habilitador da issue crítica de RLS; não é segredo.
- APP_TOKEN: falsa sensação de segurança; não é fronteira de autorização.
- .env: exposição local/backup; não há evidência de commit.

## Sugestão de correção

1. Corrigir o RLS (issue crítica) e documentar que a anon key é pública.
2. Tratar o `APP_TOKEN` como identificador de app, não segredo; manter
   `auth.uid()` + owner como barreira; rotacionar e registrar o processo.
3. Remover a `OPENAI_API_KEY` não usada do `.env`; rotacionar a
   `FLASK_SECRET_KEY` se o arquivo foi compartilhado.
4. Adicionar gitleaks no pre-commit/CI e revisar `.env.example`.

## Critérios de aceite

- [ ] A anon key não retorna dados de aluno (depende da issue de RLS).
- [ ] O APP_TOKEN não é descrito como segredo de autorização.
- [ ] Nenhuma chave OpenAI não usada permanece no `.env`.
- [ ] gitleaks roda no pre-commit/CI e o build falha em segredo detectado.
- [ ] `.env.example` contém apenas placeholders.
````

--- FIM ISSUE 9 ---

Fim do relatório. 15 achados verificados e 9 issues publicáveis. Documento gerado em 02/11/2026 por análise estática do repositório.
````